并行计算技术简介



摘要

- 为什么需要并行计算?
- 并行计算技术的分类
- 并行计算的主要技术问题
- MPI并行程序设计
- 为什么需要大规模数据并行处理?



摘要

- 为什么需要并行计算?
- 并行计算技术的分类
- 并行计算的主要技术问题
- MPI并行程序设计
- 为什么需要大规模数据并行处理?



为什么需要并行计算

- 贯穿整个计算机技术发展的核心目标：**提高计算性能！**
  - Intel CPU发展：
    - 1993 Pentium 主频60MHz，60Mflops
    - 2011 i7-2600k 主频2.93GHz，187Gflops
    - 2016 i7-6900k 主频4GHz，1023Gflops
    - 2018 i9-9900k 主频5GHz，8核16线程
    - 2020 i9-10900k 主频5.3GHz，10核20线程
  - 通过芯片制程工艺+处理器微架构设计+服务器平台技术，使CPU性能大幅提升

Intel微处理器每秒1千8百亿次浮点运算！

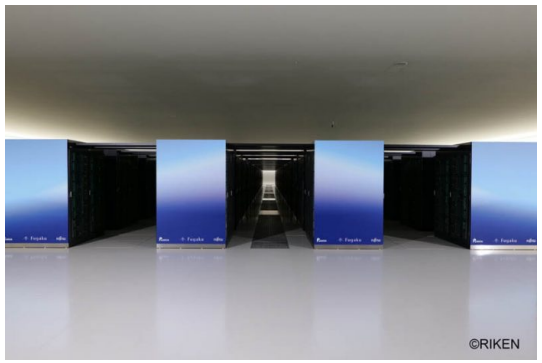
为什么需要并行计算

- 超级计算机（2023年6月）



No.1 Frontier（美国）

处理器：8,699,904个
峰值速度：1194 PFlop/s
Frontier是美国能源部橡树岭国家实验室（ORNL）推出的新超级计算机，成为全球第一款E级超算（百亿亿次），峰值运行达到第二名的三倍多。



No.2 Fugaku（日本）

处理器： 7,630,848个
峰值速度： 442.01 PFlop/s
Fugaku超算原来被称为“Post K”，是曾经的世世界第一K computer产品的第四代，采用ARM架构的富士通A64FX处理器。



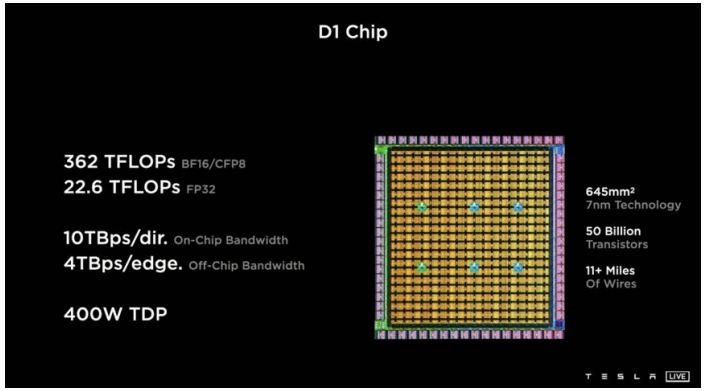
No.7 Sunway Taihulight（中国）

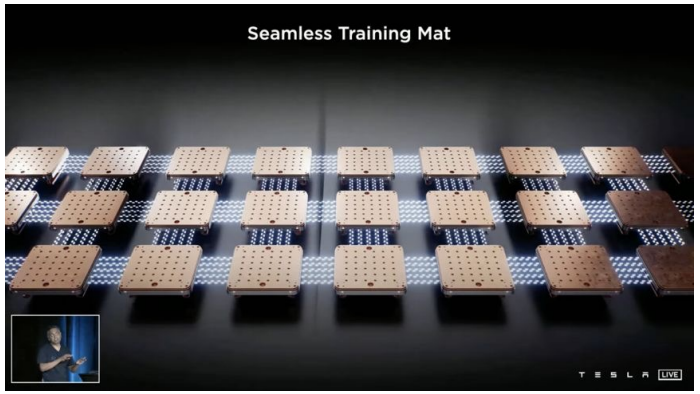
处理器：10,649,600个
峰值速度：93.01PFlop/s
神威 “太湖之光”超级计算机全部使用中国自主知识产权的芯片。

引自：https://www.chinastor.com/hpc-top500/

为什么需要并行计算

- Tesla AI Day：D1芯片与DoJo超级计算机
  - 实现人工智能训练的超高算力，同时还要扩展带宽、减少延迟、节省成本





25个D1芯片组成一个Training Tile；然后12个训练片可以组成一个服务器机柜，共108 PFlops；几个机柜再组成Dojo超级计算机。



提高计算机性能的主要手段

- 提高处理器字长：

  71年，4004， 4bits；

  78年，8086， 8bits；

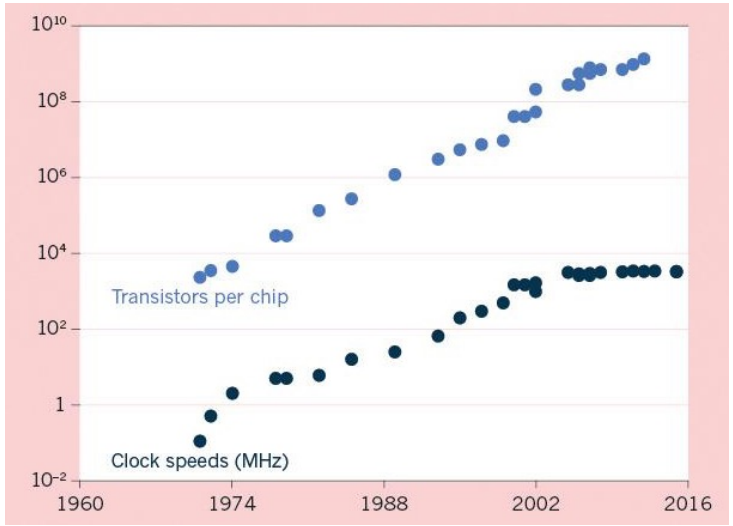
  82年，80286，16bits；

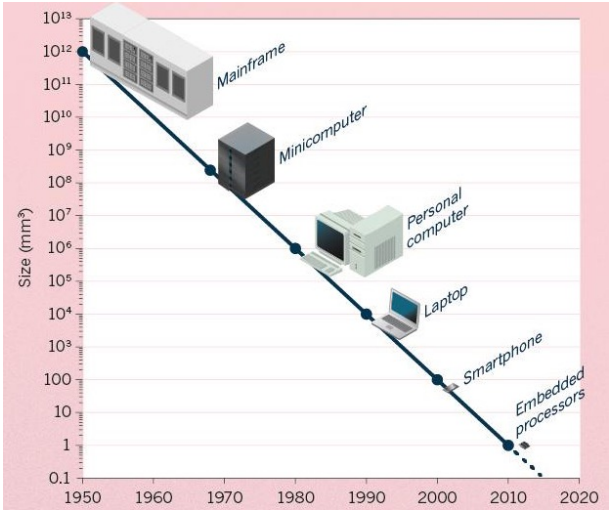
  85年~90s 80386，486，Pentium P2/P3/P4： 32bits

  05年~现在，Pentium D往后-Core i3/i5/i7/i9： 64bits

提高计算机性能的主要手段

- 提高集成度
  - 摩尔定律：芯片集成度每18个月翻一倍，计算性能提高一倍（1965年，戈登·摩尔）





挑战1：发热

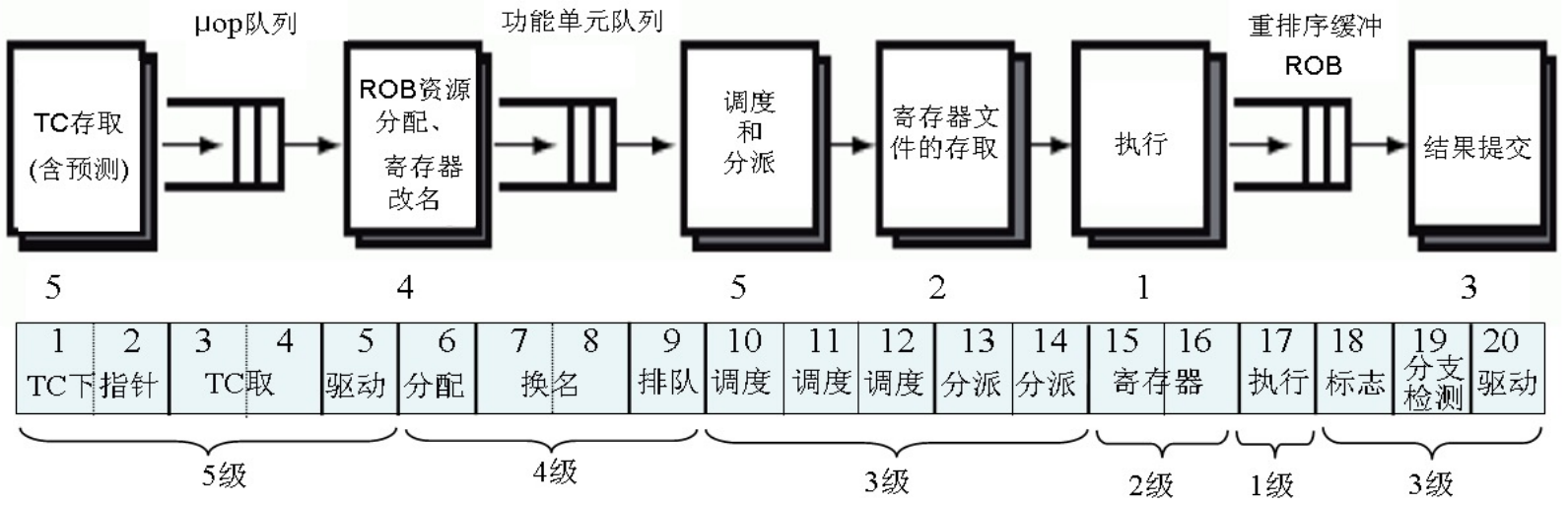
挑战2：移动化



提高计算机性能的主要手段

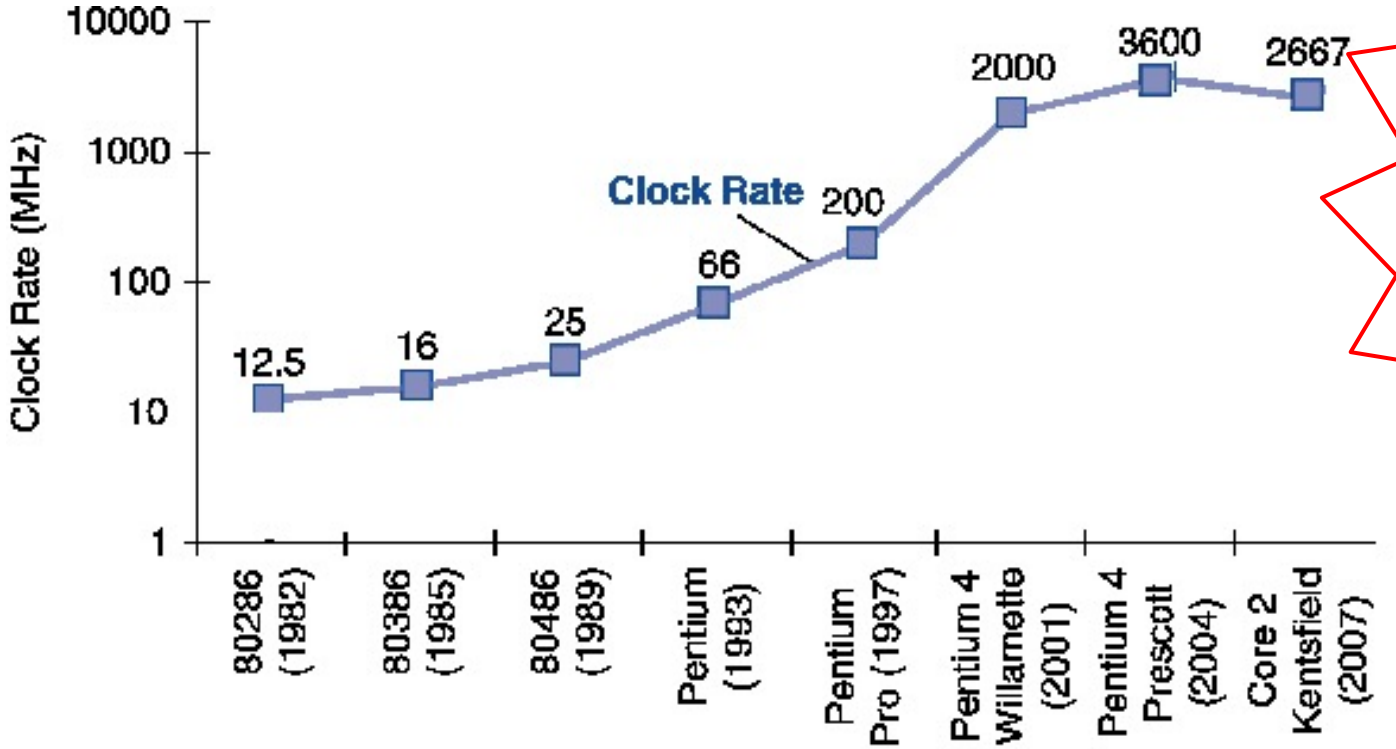
- 流水线等微体系结构技术
  - 实现指令级并行（Instruction-Level Parallelism, ILP）

RISC结构 5级流水线：分支预测，寄存器重命名，超长指令字（VLIW），超标量(Superscalar)，乱序执行，Cache……



提高计算机性能的主要手段

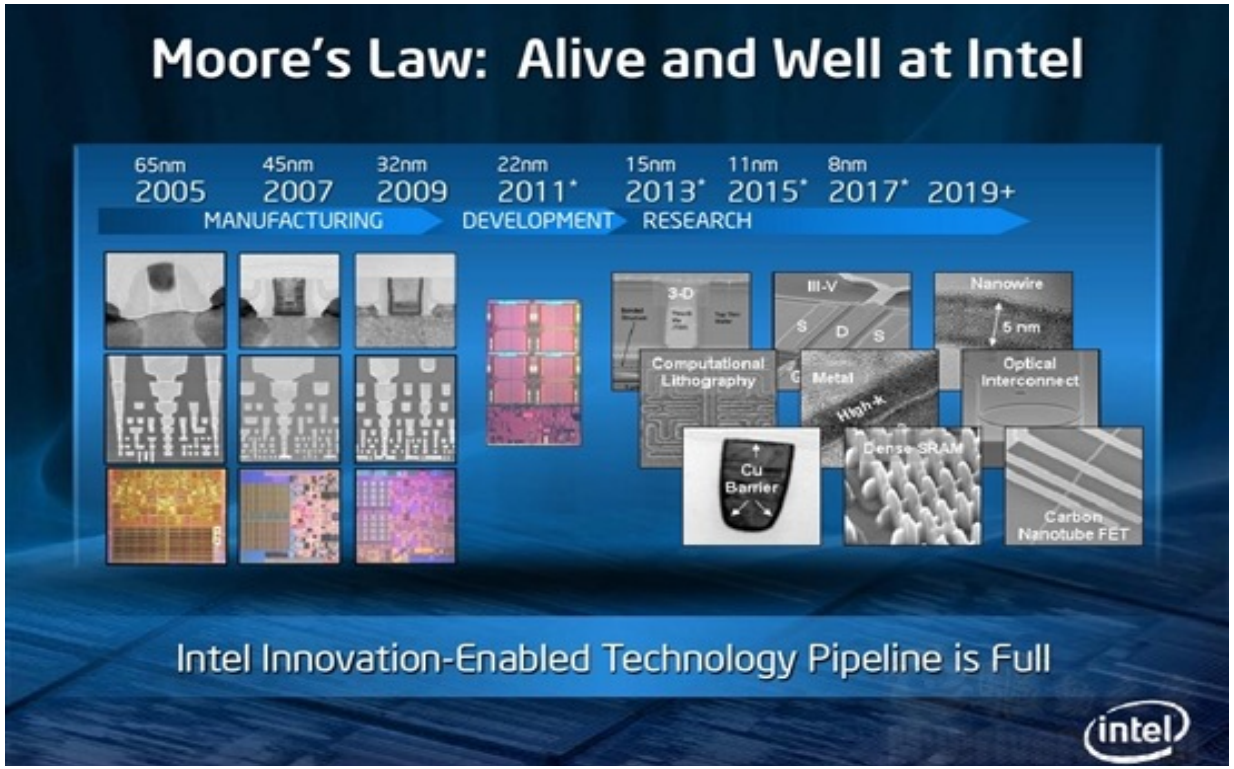
- 提高处理器频率



单核处理器性能提升接近极限！

单核处理器性能提升接近极限

- 1.VLSI集成度不可能无限制提高
  - 芯片集成度已进入极小尺度级别，集成度不可能无限制提高



1nm(纳米)
约头发直径的6万分之一
或4个原子长度

10-20nm仅有几百个原子的长度

单核处理器性能提升接近极限

- 2.处理器的指令级并行度提升接近极限

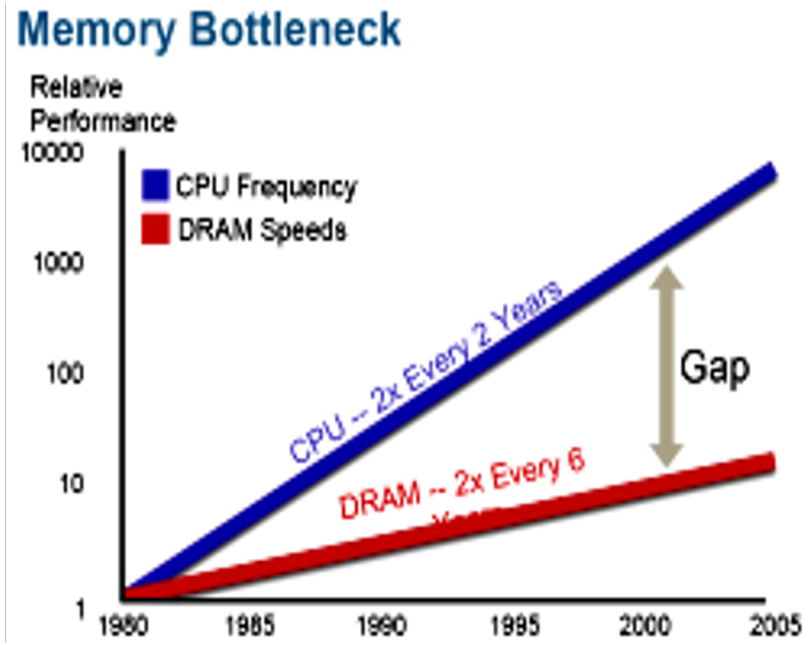
长指令字，流水线，分支预测，寄存器命名，超标量，乱序执行，动态发射，高速缓冲（Cache）……

高级流水线等各种复杂的微体系结构技术都已得到研究应用，难以进一步挖掘更多的指令级并行性（ILP）



单核处理器性能提升接近极限

- 3.处理器速度和存储器速度差异越来越大
  - 处理器性能每2年翻一倍，而存储器性能每6年翻一倍
  - 为了匹配两者间速度差异，处理器需要做越来越大的Cache



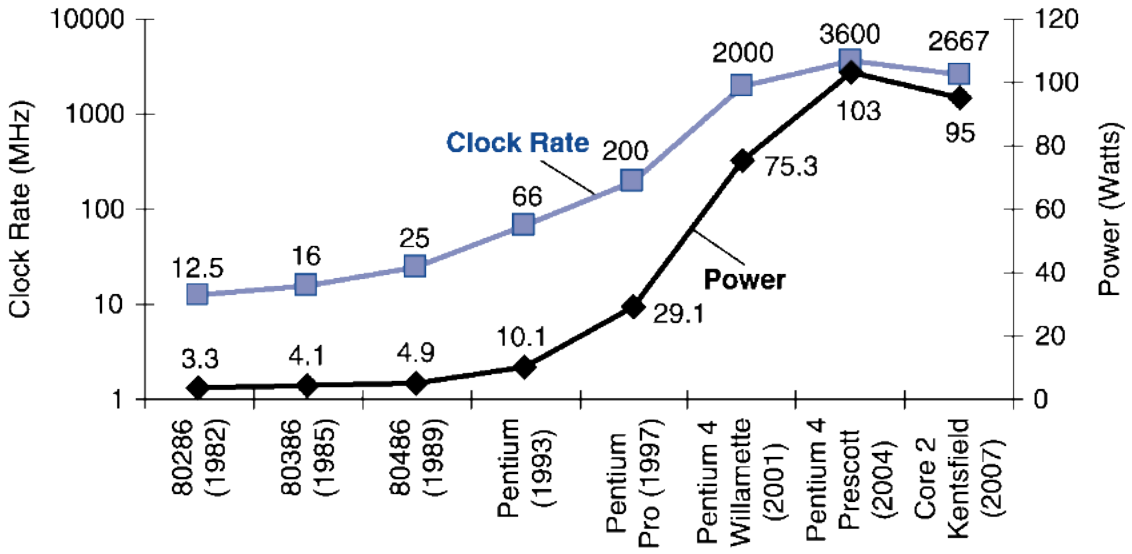
CPU计算速度：~1ns级别

主存访问速度：100ns级别

存储墙

单核处理器性能提升接近极限

- 4.功耗和散热大幅增加超过芯片承受能力
  - 晶体管密度不断提高，单位面积功耗和散热大幅增加。主频提高导致功耗和散热急剧增加
  - 功耗$P=CV^2f$，C：时钟跳变时门电路电容，V：电压，f：主频
  - 晶体管数越多，电容越大=>功耗越大；主频越高=>功耗越大

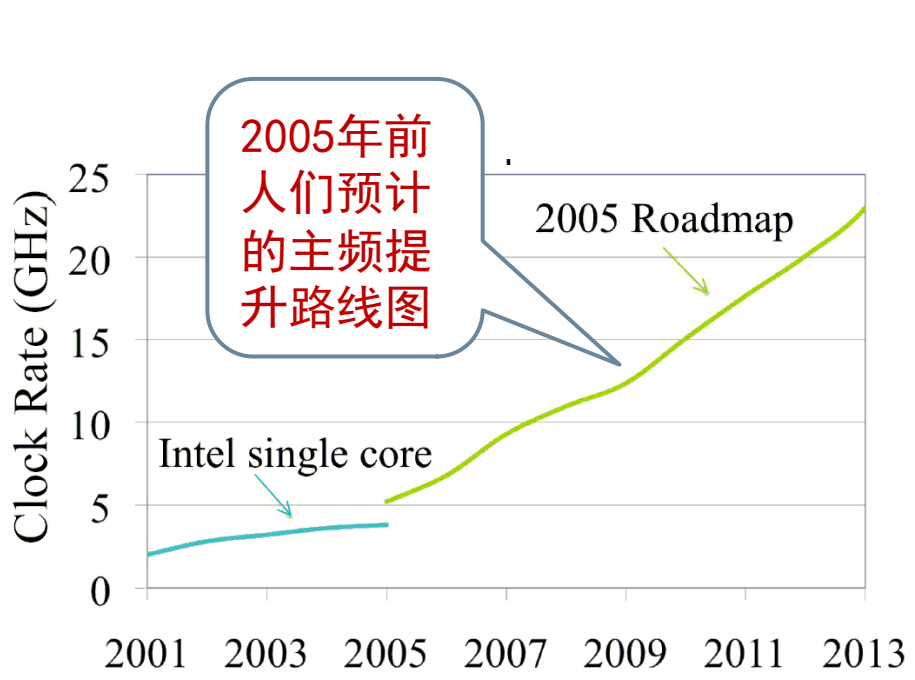


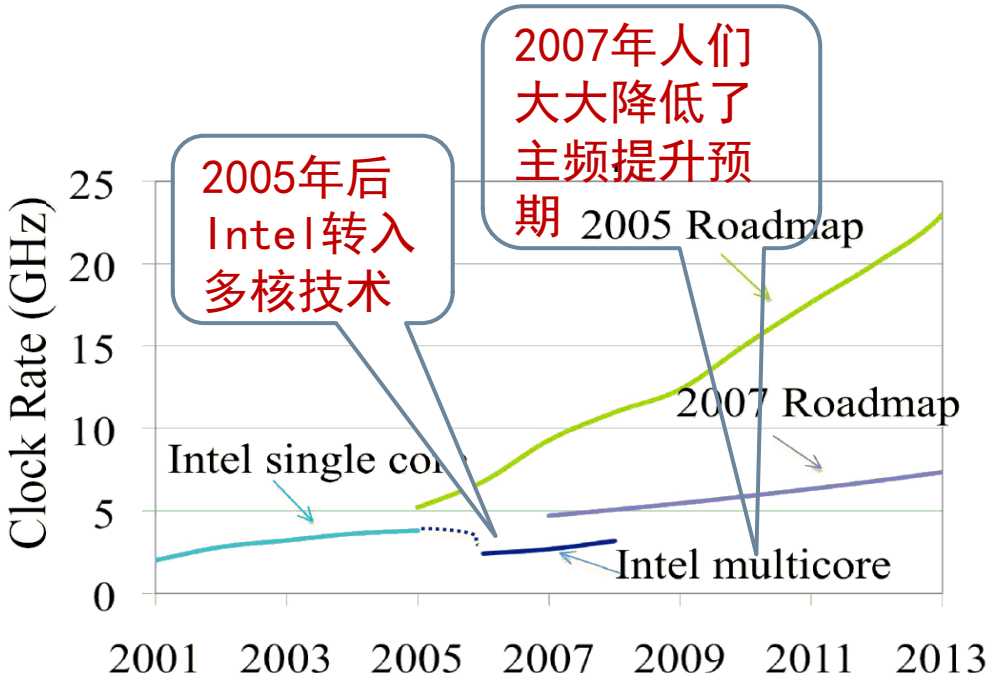


Cite from Edward L. Bosworth, The Power Wall, 2010

单核处理器性能提升接近极限

- 2005年前，人们预期可以一直提升处理器主频
- 但2004年5月Intel处理器Tejas and Jayhawk(4GHz)因无法解决散热问题最终放弃，标志着升频技术时代的终结



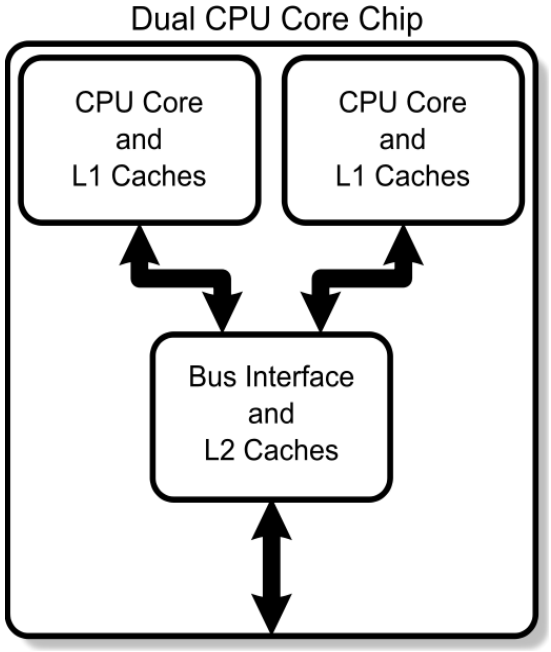


Cite from Edward L. Bosworth, The Power Wall, 2010

向多核并行计算发展成为必然趋势

多核/众核并行计算

- 2005年Intel全面转入多核计算技术，采用多核/众核构架，简化单处理器的复杂设计，代之以单个芯片上设计多个简化的处理器核，以多核/众核并行计算提升计算性能



- 双核：Pentium D, EE, Xeon Core 2 Duo E系列, T系列, Core i3, i5
- 4核： Core 2 Quad Q Core i5, i7
- 6核： Core i7 970/980/…
- 8核：AMD Bulldozer
- 10核： Core i9-7900X

典型的双核处理器结构

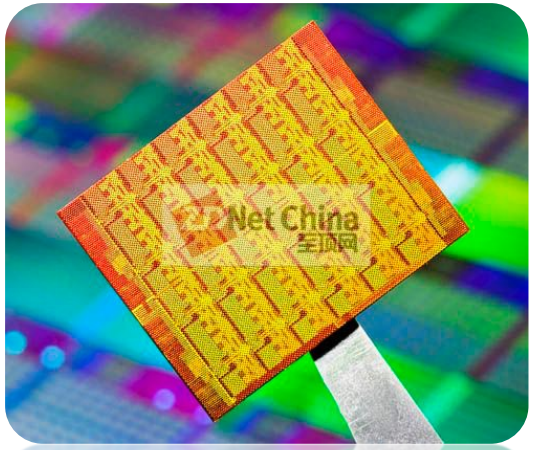
向多核并行计算发展成为必然趋势

- 多核/众核并行计算

  Intel实验芯片

  Single Cloud Chip, SCC：48核

  Teraflops，80核



- ASCI Red：1996，第一个达到1TFlops(10万亿次浮点运算)的并行计算系统，使用了10,000颗Pentium Pro处理器(200MHz)，耗电500kW，外加500kW用于机房散热
- Teraflops：达到1.01TFlops(3.16GHz) 1.81TFlops(5.7GHz)，功耗62W!

Cite from Intel website:
http://techresearch.intel.com/projectdetails.aspx?id=151

向多核并行计算发展成为必然趋势

- 多核/众核并行计算
  - 根据摩尔定律，Intel预计其通用的众核并行计算芯片
  - 2015年：128核
  - 2017年：256核
  - 2019年：512核
  - 2023年：2048核

NVIDIA GPU

Graphic Processing Unit，主要用于图形图像并行处理

Tesla M2050/2070: 448核

S2050 1U GPU 处理系统: 4个M2050/2070，1792核





Functionality	Xeon Phi 5110p	Tesla k20x	Firepro S10000
Gpu-processor	1	1	2
Memory	8GB GDDR5	6GB GDDR5	3GB GDDR5 *
Memory Bandwidth	320 GB/s	200 GB/s	240 GB/s *
SP performance	2,022 TFLOPS	3,95 TFLOPS	2,95 TFLOPS *
DP performance	1,011 TFLOPS	1,31 TFLOPS	0,74 TFLOPS *
Max Power usage	225 Watt	235 Watt	325 Watt

向多核并行计算发展成为必然趋势

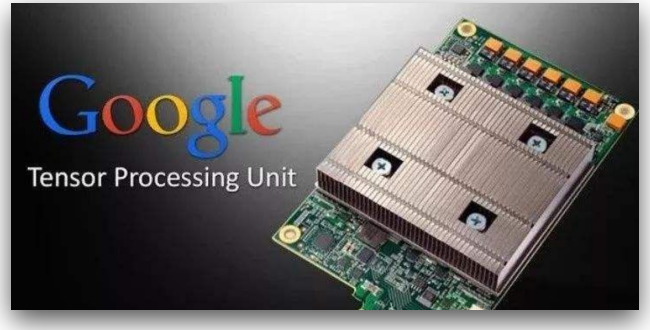
- 多核/众核并行计算
  - AI时代又进一步催生了TPU、NPU等众核AI处理器的诞生
    - 黄仁勋提出的Huang‘s Law——GPU将推动AI性能实现逐年翻倍

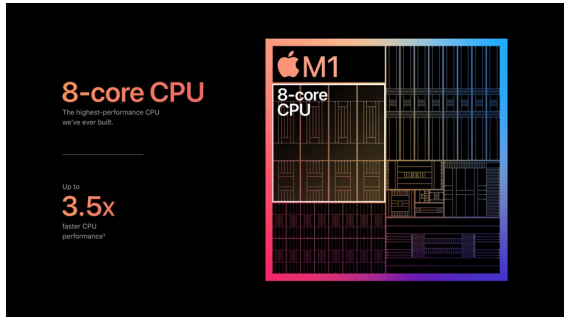
CPU
- Small models
- Small datasets
- Useful for design space exploration

GPU
- Medium-to-large models, datasets
- Image, video processing
- Application on CUDA or OpenCL

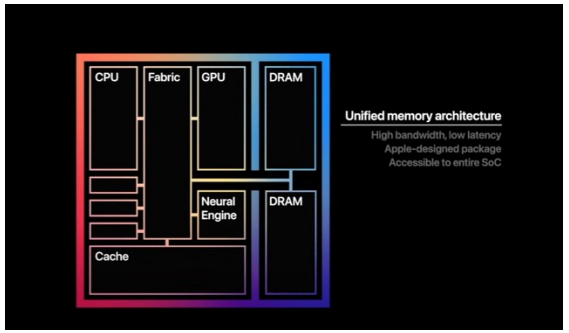
TPU
- Matrix computations
- Dense vector processing
- No custom TensorFlow operations

FPGA
- Large datasets, models
- Compute intensive applications
- High performance, high perf./cost ratio









应用领域计算规模和复杂度大幅提高

- 爆炸性增长的Web规模数据量
  - Google从2004年每天处理100TB数据到2008年每天处理20PB
  - 2009年eBays数据仓库，一个有2PB用户数据，另一个6.5PB用户数据包含170TB记录且每天增长150GB个记录；Facebook：2.5PB用户数据，每天增加15TB
  - 世界最大电子对撞机每年产生15PB(1千5百万GB)数据
  - 2015年落成的世界最大观天望远镜主镜头像素为3.2G，每年将产生6PB天文图像数据；
  - 欧洲生物信息研究中心(EBI)基因序列数据库容量已达5PB；中国深圳华大基因研究所成为全世界最大测序中心，每天产生300GB基因序列数据（每年100TB）

应用领域计算规模和复杂度大幅提高

- 超大的计算量/计算复杂度
  - 用SGI工作站进行电影渲染时，每帧一般需要1～2小时
  - 一部2小时的电影渲染需要：

    2小时x3600秒x24帧x(1~2小时)/24小时=20~40年!
  - 特殊场景每帧可能需要60个小时（影片《星舰骑兵》中数千只蜘蛛爬行的场面），用横向4096象素分辨率进行渲染时，如果以每帧60个小时的速度，则1秒的放映量（24帧）需要60天的渲染时间，1分钟则需要10年!
  - 世界著名的数字工作室Digital Domain公司用了一年半的时间，使用了300多台SGI超级工作站，50多个特技师一天24小时轮流制作《泰坦尼克号》中的电脑特技

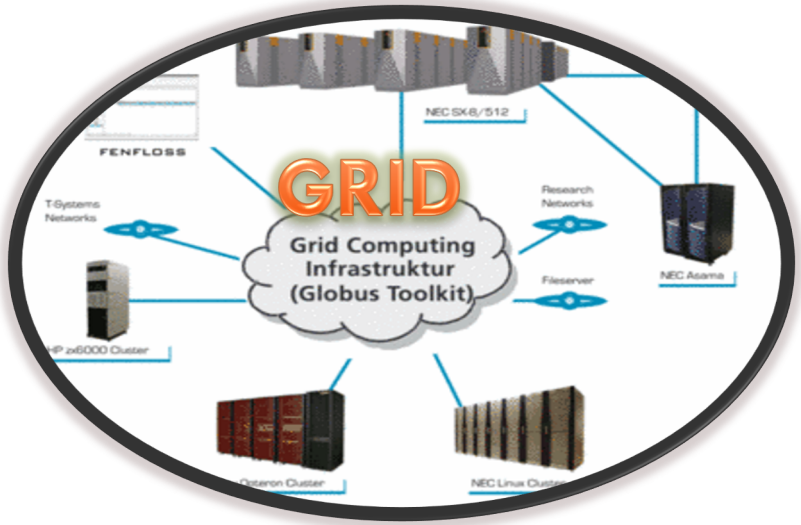
解决方案——并行计算！


Multicore


Manycore
SMP


MPP


Cluster


GRID
Grid Computing
Infrastruktur
(Globus Toolkit)
NEC SX-6/512
FENFLOSS
T-Systems Networks
Research Networks
Fileserver
NEC Asama
Opteron Cluster
NEC Linux Cluster


Cloud



并行计算技术的发展趋势和影响

- 越来越多的研究和应用领域需要使用并行计算技术
  - 并行计算技术将渗透到每个计算应用领域，尤其是涉及到大规模数据和复杂计算的应用领域
- 并行计算技术将对传统计算技术产生革命性的影响
  - 并行计算技术将影响传统计算技术的各个层面，与传统计算技术相互结合产生很多新的研究热点和课题
  - 很多传统的串行算法和计算方法都将需要重新研究和设计其并行化算法和计算方法



为什么需要学习并行计算技术?

- 软件开发/程序设计人员面临挑战!
  - 20-30年里程序设计技术的最大的革命是面向对象技术
  - 下一个程序设计技术的革命将是并行程序设计
  - 今天绝大多数程序员不懂并行设计技术，就像15年前绝大多数程序员不懂面向对象技术一样

Cite from Herb Sutter, The Free Lunch Is Over-A Fundamental Turn Toward Concurrency in Software , Dr. Dobb's Journal, 30(3), March 2005



摘要

- 为什么需要并行计算?
- 并行计算技术的分类
- 并行计算的主要技术问题
- MPI并行程序设计
- 为什么需要大规模数据并行处理?



并行计算技术的分类

- 按数据和指令处理结构：弗林(Flynn)分类
- 按并行类型
- 按存储访问构架
- 按系统类型
- 按计算特征
- 按并行程序设计模型/方法

弗林分类

- SISD：单指令单数据流

  传统的单处理器串行处理

- SIMD：单指令多数据流

  向量机，信号处理系统

- MISD：多指令单数据流

  很少使用

- MIMD：多指令多数据流

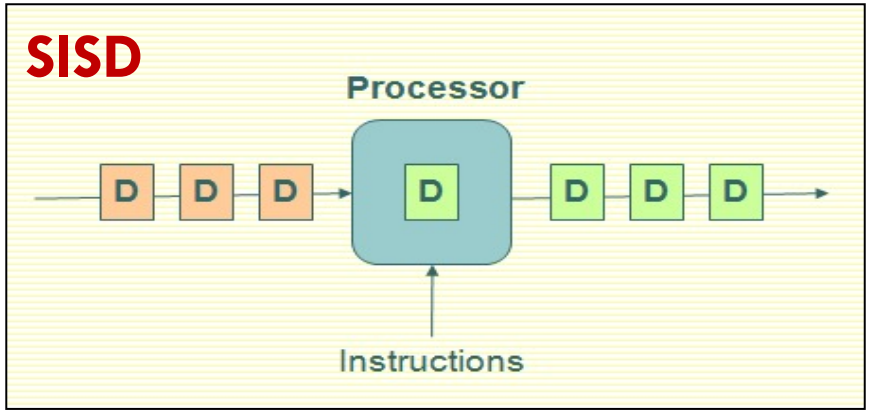
  最常用，TOP500

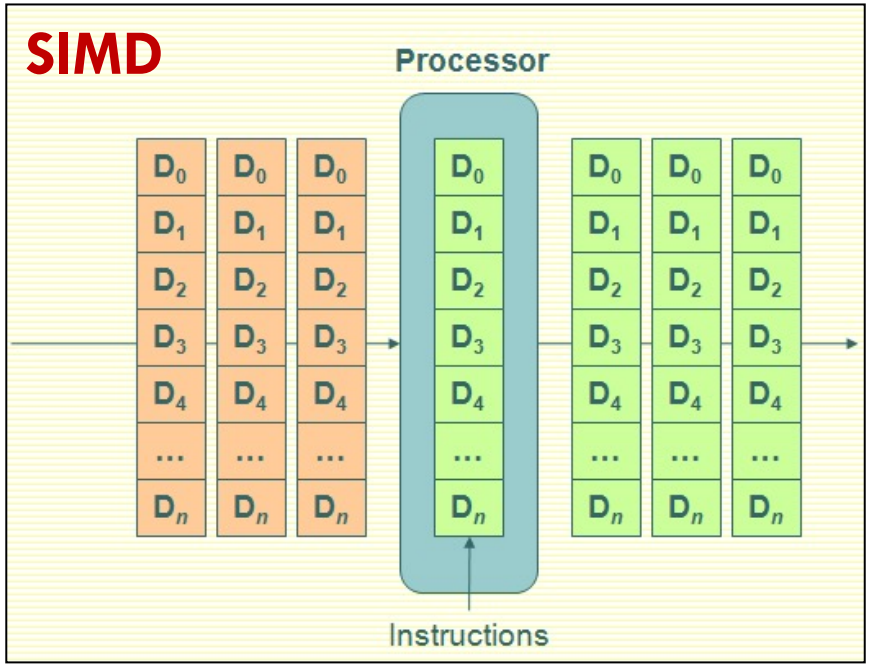
  基本都属于MIMD类型

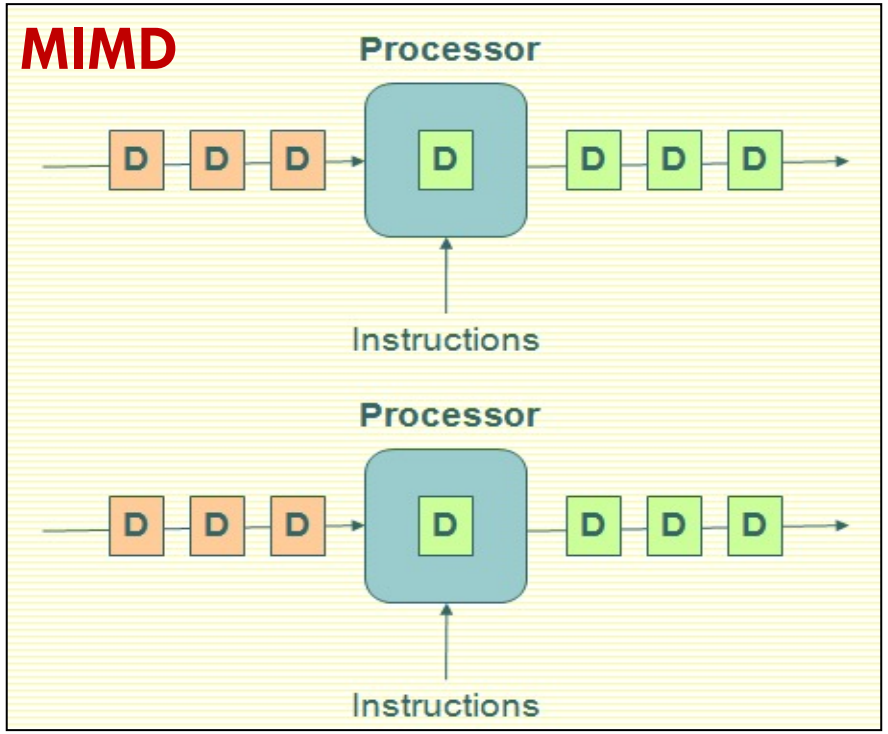
Flynn's taxonomy

	Single Instruction	Multiple Instruction
Single Data	SISD	MISD
Multiple Data	SIMD	MIMD

弗林分类







Cite from Jimmy Lin, What is cloud computing, 2008

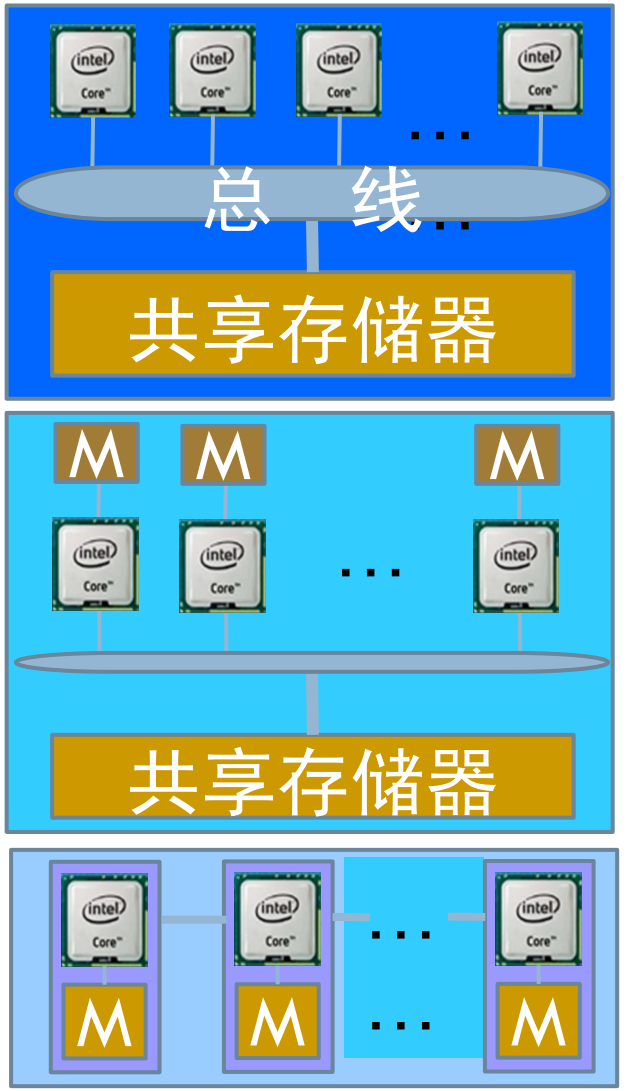


按并行类型分类

- 位级并行（Bit-Level Parallelism）
- 指令级并行（ILP：Instruction-Level Parallelism）
- 线程级并行（Thread-Level Parallelism）
  - 数据级并行：一个大的数据块划分为小块，分别由不同的处理器/线程处理
  - 任务级并行：一个大的计算任务划分为子任务，分别由不同的处理器/线程来处理

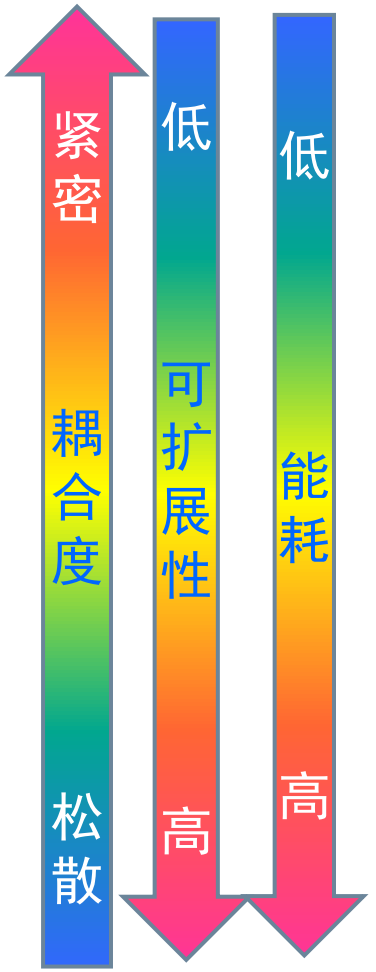
按存储访问结构分类

- 共享内存（Shared Memory) – UMA结构 (Uniform Memory Access)
  - 所有处理器通过总线共享内存
  - 多核处理器，SMP……
- 分布共享存储体系结构 – NUMA结构 (Non-Uniform Memory Access)
  - 各个处理器有本地存储器
  - 同时再共享一个全局的存储器
- 分布式内存 – NUMA结构
  - 各个处理器使用本地独立的存储器



按系统类型分类

- 多核/众核并行计算系统MC （Multi-Core）
  - 或Chip-level multiprocessing, CMP
- 对称多处理系统SMP（Symmetric Multi Processing）
  - 多个相同类型处理器通过总线连接并共享存储器
- 大规模并行处理MPP （Massively Parallel Processing）
  - 专用内联网连接一组处理器形成的一个计算系统
- 集群Cluster
  - 网络连接的一组商品计算机构成的计算系统
- 网格Grid
  - 用网络连接远距离分布的一组异构计算机构成的计算系统
- 云Cloud
  - 通过互联网按需访问计算资源





按系统类型分类

- 不同系统的特征和对比
  - 从MC到Cloud，耦合度越来越低，但可扩展性越来越高，系统规模越来越大，而能耗也越来越高
    - MC处理器核通过NOC(片上网络)集成在一个芯片上，通常使用混合式内存访问机制(本地缓存加全局内存)，功耗很低
    - SMP使用独立的处理器和共享内存，以总线结构互联，运行一个操作系统，定制成本高,难以扩充,规模较小(2-8处理器)
    - MPP使用独立的处理器及独立的内存、OS，专用的高速内联网络，难以升级和扩充，规模中等



按系统类型分类

- 不同系统的特征和对比
  - 从MC到Cloud，耦合度越来越低，但可扩展性越来越高，系统规模越来越大，而能耗也越来越高
    - Cluster使用商品化的刀片或机架服务器，以网络互联为一个物理上紧密的计算系统，可扩展性强，规模可小可大，是目前高性能并行计算最常用的形式
    - Grid则为地理上广泛分布的异构计算资源构成的一个极为松散的计算系统，主要用于并行度很低的大规模科学计算任务
    - Cloud云计算就是通过互联网按需访问计算资源，即应用程序、服务器（物理服务器和虚拟服务器）、数据存储、开发工具、网络功能等，这些资源托管在由云服务提供商管理的远程数据中心上。



按计算特征分类

- 数据密集型并行计算 (Data-Intensive Parallel Computing)
  - 数据量极大、但计算相对简单的并行处理，如大规模Web 信息搜索
- 计算密集型并行计算(Computation-Intensive Parallel Computing)
  - 数据量相对不是很大、但计算较为复杂的并行处理，如：3-D建模与渲染，气象预报，科学计算……
- 数据密集与计算密集混合型并行计算
  - 兼具数据密集型和计算密集型特征的并行计算，如3D电影渲染



按并行程序设计模型/方法分类

- 共享内存变量 (Shared Memory Variables)
  - 多线程共享存储器变量方式进行并行程序设计，会引起数据不一致性，导致数据和资源访问冲突，需要引入同步控制机制；Pthread，OpenMP：共享内存式多处理并行编程接口
- 消息传递方式(Message Passing)
  - 对于分布式内存结构，为了分发数据和收集计算结果，需要在各个计算节点间进行数据通信，最常用的是消息。
  - 传递方式；MPI：消息传递并行编程接口标准
- MapReduce方式
  - Google公司提出的MapReduce并行程序设计模型，是当时最易于使用的并行程序设计方法，广泛使用于搜索引擎等大规模数据并行处理



发展历史和现状

- 1975-1985
  - 主要是向量机技术，如Cray1，Cray2。但基于多线程的并行计算也逐步引入。
- 1986-1995
  - 大规模并行处理MPP成为主流并行计算技术，消息传递编程接口MPI得到开发应用。目前TOP500中有84个基于MPP。
- 1995-现在
  - Cluster和Grid并行计算技术成为主流，但目前Grid的发展已呈下降趋势，目前TOP500中有414个基于Cluster。



发展趋势

- SMP作为共享内存式小规模并行计算技术一直活跃
  - 60-70年代基于大型机的SMP系统，80年代基于80386 /80486的SMP系统，90年代到目前基于多核的个人电脑、服务器大都基于SMP
- 多核/众核并行计算成为重要发展趋势
  - 由于单核处理器性能发展的瓶颈，同时由于多核/众核计算计算自身具有的体积小、功耗低等诸多技术特点和优势，今后多核/众核并行计算会称为必然趋势
- 并行计算软件技术远远落后于硬件发展速度
  - 并行计算硬件技术水平和规模发展迅速，但并行计算软件技术远远跟不上硬件发展水平和速度，缺少有效的并行计算软件框架、编程模型和方法

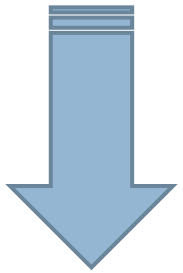


摘要

- 为什么需要并行计算?
- 并行计算技术的分类
- 并行计算的主要技术问题
- MPI并行程序设计
- 为什么需要大规模数据并行处理?

并行计算的主要技术问题

数据怎么存？怎么算？



并行算法

软件构架

硬件构架



并行计算的主要技术问题

- 多核/多处理器网络互连结构技术
- 存储访问体系结构
- 分布式数据与文件管理
- 并行计算任务分解与算法设计
- 并行程序设计模型和方法
- 数据同步访问和通信控制
- 可靠性设计与容错技术
- 并行计算软件框架平台
- 系统性能评价和程序并行度评估



多核/多处理器网络互联结构技术

- 主要研究处理器间互联拓扑结构，尤其在包含大量处理器的并行计算系统中，需要具有良好的互联结构，以保证大量处理器能真正有效地协同工作，获得应有的并行计算效率。
  - 共享总线连接（Shared Bus）
  - 交叉开关矩阵（Crossbar Switch）
  - 环形结构（Torus）
  - Mesh网络结构（Mesh Network）
  - 片上网络（NOC，Network-on-chip）
  - ……



存储访问体系结构

- 主要研究不同的存储结构，以及在不同存储结构下的特定技术问题
  - 共享存储器体系结构(Shared Memory)
    - 共享数据访问与同步控制
  - 分布存储体系结构(Distributed Memory)
    - 数据通信控制和节点计算同步控制
  - 分布共享存储结构(Distributed Shared Memory)
    - Cache的一致性问题
    - 数据访问/通信的时间延迟



分布式数据与文件管理

- 并行计算的一个重要问题是，在大规模集群环境下，如何解决大数据块的划分、存储和访问管理；尤其是数据密集型并行计算时，理想的情况是提供分布式数据与文件管理系统，如：
  - RedHat GFS (Global File System)
  - IBM GPFS
  - Sun Lustre
  - Google GFS(Google File System)
  - Hadoop HDFS(Hadoop Distributed File System)



并行计算任务的分解与算法设计

- 一个大型计算任务如何从数据上或者是计算方法上进行适当的划分，分解为一组子任务以便分配给各个节点进行并行处理，如何搜集各节点计算的局部结果。
  - 数据划分
    - 如何将特大的数据进行划分并分配给各节点进行处理
  - 算法分解与设计
    - 一个大的尤其是计算密集型的计算任务，首先需要寻找并确定其可并行计算的部分，然后进一步寻找好的分解算法：可把一个整体的算法纵向分解为一组并行的子任务，或者对于复杂的计算任务可横向分解为多次并行处理过程。



并行程序设计模型和方法

- 根据不同的硬件构架，不同的并行计算系统可能需要不同的并行程序设计模型、方法、语言和编译技术。
- 并行程序设计模型和方法
  - 共享内存式并行程序设计：为共享内存结构并行计算系统提供的程序设计方法,需提供数据访问同步控制机制(如互斥信号,锁等)
  - 消息传递式并行程序设计：为分布内存结构并行计算系统提供的、以消息传递方式完成节点间数据通信的程序设计方法
  - MapReduce并行程序设计：为解决前两者在并行程序设计上的缺陷，提供一个综合的编程框架，为程序员提供了一种简便易用的并行程序设计方法



并行程序设计模型和方法

并行程序设计语言

- 语言级扩充：使用宏指令在普通的程序设计语言（如C语言）上增加一些并行计算宏指令，如 OpenMP（提供C，C++，Fortran语言扩充，Linux & Windows）
- 并行计算库函数与编程接口：使用函数库提供并行计算编程接口，如MPI(消息传递接口)，CUDA(NVIDIA GPU)

```
int main(int argc, char *argv[]) {
    const int N = 100000;  int i,
a[N];
    #pragma omp parallel for
    for (i = 0; i < N; i++)
        a[i] = 2 * i;
    return 0;
}
```

并行编译与优化技术

- 编译程序需要考虑编译时的自动化并行性处理，以及为提高计算性能进行并行计算优化处理



数据同步访问和通信控制

- 如何解决并行化计算中共享数据访问和节点数据通信问题
  - 共享数据访问和同步控制
    - 在包含共享存储器结构的系统中，不同处理器/线程访问共享存储区时，可能会导致数据访问的不确定性（竞争状态，race condition），因此需要考虑使用同步机制（互斥信号，条件变量等）保证共享数据和资源访问的正确性，还要解决同步可能引起的死锁问题。
  - 分布存储结构下的数据通信和同步控制
    - 在包含分布存储器结构的系统中，不同处理器/线程需要划分和获取计算数据，这些数据通常需要由主节点传送到各个从节点；由于各个节点计算速度不同，为了保证计算的同步，还需要考虑各节点并行计算的同步控制（如Barrier，同步障）



可靠性设计与容错技术

- 大型并行计算系统使用大量计算机,因此,节点出错或失效是常态,不能因为一个节点失效导致数据丢失、程序终止或系统崩溃，因此，系统需要具有良好的可靠性设计和有效的失效检测和恢复技术。
  - 数据失效恢复：大量的数据存储在很多磁盘中，当出现磁盘出错和数据损坏时，需要有良好的数据备份和数据失效恢复机制，保证数据不丢失以及数据的正确性。
  - 系统和任务失效恢复：一个节点失效不能导致系统崩溃，而且要能保证程序的正常运行，因此，需要有很好的失效检测和隔离技术，并进行计算任务的重新调度以保证计算任务正常进行。



并行计算软件框架平台

- 提供自动化并行处理能力
  - 现有的OpenMP、MPI、CUDA等并行程序设计方法需要程序员考虑和处理数据存储管理、数据和任务划分和调度执行、数据同步和通信、结果收集、出错恢复处理等几乎所有技术细节，非常繁琐
  - 需要研究一种具有自动化并行处理能力的并行计算软件框架和平台，提供自动化的并行处理，能屏蔽并行化处理的诸多系统底层细节，交由软件框架来处理，提供必要的编程接口，简化程序员的编程，让程序员从系统底层细节中解放出来，专注于应用问题本身的计算和算法的实现。如Google和Hadoop MapReduce
- 高可扩展性和系统性能提升
  - 并行计算框架允许方便地增加节点扩充系统，但系统节点的增加不影响程序的编写，并且要能保证节点增加后系统性能有线性的提升

系统性能评估和程序并行度评估

- 系统性能评估
  - 用标准性能评估(Benchmark)方法评估一个并行计算系统的浮点计算能力。High-Performance Linpack Benchmark是最为知名的评估工具，TOP500用其进行评估和排名
- 程序并行度评估
  - 程序能得到多大并行加速依赖于该程序有多少可并行计算的比例。经典的程序并行加速评估公式Amdahl定律：

$$S=\frac{1}{(1-P)+\frac{P}{N}}$$

其中，S是加速比，P是程序可并行比例，N是处理器数目

系统性能评估和程序并行度评估

根据Amdahl定律：

一个并行程序可加速程度是有限制的，并非可无限加速，并非处理器越多越好

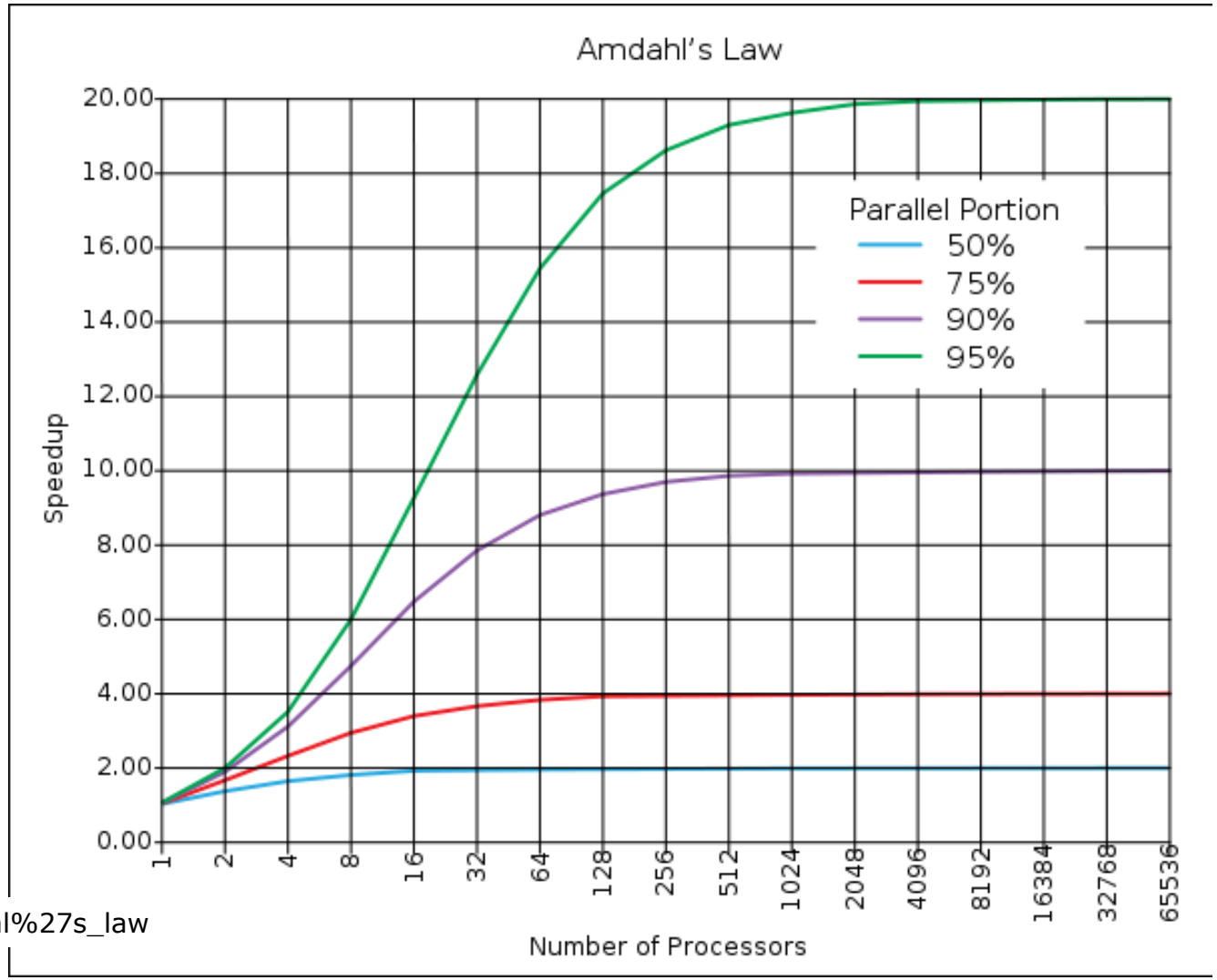
并行比例 vs 加速比

50%=>最大2倍

75%=>最大4倍

90%=>最大10倍

95%=>最大20倍



Cite from http://en.wikipedia.org/wiki/Amdahl%27s_law



摘要

- 为什么需要并行计算?
- 并行计算技术的分类
- 并行计算的主要技术问题
- **MPI并行程序设计**
- 为什么需要大规模数据并行处理?



MPI并行程序设计

- MPI (Message Passing Interface)，基于消息传递的高性能并行计算编程接口
- 在处理器间以消息传递方式进行数据通信和同步，以库函数形式为程序员提供了一组易于使用的编程接口。
- 93年由一组来自大学、国家实验室、高性能计算厂商发起组织和研究，94年公布了最早的版本MPI 1.0，MPICH是MPI最流行的非专利实现，目前（2023年）版本是MPICH-4.1.2
- 特点：提供可靠的、面向消息的通信；在高性能科学计算领域广泛使用，适合于处理计算密集型的科学计算；独立于语言的编程规范，可移植性好



MPI实现版本

厂商实现

HP-MPI	Hewlett Packard; Convex SPP
MPI-F	IBM SP1/SP2
Hitachi/MPI	Hitachi
SGI/MPI	SGI PowerChallenge series
MPI/DE	NEC.
INTEL/MPI	Intel. Paragon (iCC lib)
T.MPI	Telmat Multinode
Fujitsu/MPI	Fujitsu AP1000
EPCC/MPI	Cray & EPCC, T3D/T3E

语言实现

C/C++
Java
Python
.NET

开放领域/机构实现

MPICH 阿贡国家实验室和密西西比大学
最早的完整MPI标准实现.

LAM Ohio Supercomputer center
MPICH/NT Mississippi State University
MPI-FM Illinois (Myrinet)
MPI-AM UC Berkeley (Myrinet)
MPI-PM RWCP, Japan (Myrinet)
MPI-CCL California Institute of Technology
CRI/EPCC MPI Cray Research and Edinburgh Parallel Computing Centre
MPI-AP Australian National University-CAP Research Program (AP1000)
W32MPI Illinois, Concurrent Systems
RACE-MPI Hughes Aircraft Co.
MPI-BIP INRIA, France (Myrinet)



MPI并行程序设计

- 消息传递并行程序设计
  - 用户必须通过显式地发送和接收消息来实现处理机间的数据交换。
  - 每个并行进程均有自己独立的地址空间，相互之间访问不能直接进行，必须通过显式的消息传递来实现。
- 并行计算粒度大，特别适合于大规模可扩展并行算法
  - 要求用户很好地分解问题，组织不同进程间的数据交换，并行计算粒度大。



MPI主要功能

- 用常规语言编程方式，所有节点运行同一个程序，但处理不同的数据
- 提供点对点通信(Point-point communication)
  - 提供同步通信功能（阻塞通信）
  - 提供异步通信功能（非阻塞通信）
- 提供节点集合通信(Collective communication)
  - 提供一对多的广播通信
  - 提供多节点计算同步控制
  - 提供对结果的规约(Reduce)计算功能
- 提供用户自定义的复合数据类型传输



MPI基本程序结构


```
#include <mpi.h>

main(int argc, char **argv)
{
   int numtasks, rank;

   MPI_Init(&argc, &argv);

   ……

   并行计算程序体
   ……

   MPI_Finalize();

   exit(0);
}
```

MPI程序头文件

初始化MPI环境

并行计算与通信

关闭MPI环境



MPI并行程序设计接口

- 基本编程接口：MPI提供了6个最基本的编程接口，理论上任何并行程序都可以通过这6个基本API实现

1. MPI_Init (argc, argv)： 初始化MPI，开始MPI并行计算程序体
2. MPI_Finalize: 终止MPI并行计算
3. MPI_Comm_Size(comm, size): 确定指定范围内处理器/进程数目
4. MPI_Comm_Rank(comm, rank)：确定一个处理器/进程的标识号
5. MPI_Send (buf, count, datatype, dest, tag, comm)：发送一个消息
6. MPI_Recv (buf, count, datatype, source, tag, comm, status)：接受消息

- size: 进程数，rank：指定进程的ID
- comm：指定一个通信组(communicator)
- dest：目标进程号，source：源进程标识号，tag：消息标签

MPI并行程序设计接口

- MPI并行计算初始化与结束
  - 任何一个MPI程序都要用MPI_Init和MPI_Finalize来指定并行计算开始和结束的地方；同时在运行时，这两个函数将完成MPI计算环境的初始化设置以及结束清理工作。处于两者之间的程序即被认为是并行化的，将在每个机器上被执行。

```
#include <mpi.h>
#include <stdio.h>
main(int argc, char **argv)
{
   int numtasks, rank;
   MPI_Init(&argc, &argv);
   printf("Hello parallel
            world!\n");
   MPI_Finalize();
   exit(0);
}
```

在一个有5个处理器的系统中，输出为：

```
Hello parallel world!
Hello parallel world!
Hello parallel world!
Hello parallel world!
Hello parallel world!
```



MPI并行程序设计接口

- 通信组（Communicator）
  - 为了在指定的范围内进行通信，可以将系统中的处理器划分为不同的通信组；一个处理器可以同时参加多个通信组；MPI定义了一个最大的缺省通信组：MPI_COMM_WORLD，指明系统中所有的进程都参与通信。一个通信组中的总进程数可以由MPI_Comm_Size调用来确定。
- 进程标识
  - 为了在通信时能准确指定一个特定的进程，需要为每个进程分配一个进程标识，一个通信组中每个进程标识号由系统自动编号（从0开始）；进程标识号可以由MPI_Comm_Rank调用来确定。



MPI并行程序设计接口

- 点对点通信
  - 同步通信：阻塞式通信，等待通信操作完成后才返回
    - *MPI_Send (buf, count, datatype, dest, tag, comm)*：发送一个消息
    - *MPI_Recv (buf, count, datatype, source, tag, comm, status)*：接受消息
  - 同步通信时一定要等到通信操作完成，这会造成处理器空闲，因而可能导致系统效率下降，为此MPI提供异步通信功能
  - 异步通信：非阻塞式通信，不等待通信操作完成即返回
    - *MPI_ISend (buf, count, datatype, dest, tag, comm, request)*：异步发送
    - *MPI_IRecv (buf, count, datatype, source, tag, comm, status, request)* 异步接受消息
    - *MPI_Wait (request, status)*：等待非阻塞数据传输完成
    - *MPI_Test (request, flag, status)*：检查是否异步数据传输确实完成



MPI编程示例：简单示例


```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char **argv)
{
   int num, rk;
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &num);
   MPI_Comm_rank(MPI_COMM_WORLD, &rk);

   printf("Hello world from Process %d of %d\n", rk, num);

   MPI_Finalize();
}
```

```
Hello world from Process 0 of 5
Hello world from Process 1 of 5
Hello world from Process 2 of 5
Hello world from Process 3 of 5
Hello world from Process 4 of 5
```



MPI编程示例：消息传递示例1

发送/接收消息：Hello World

```
#include <stdio.h>
#include <mpi.h>
int main(int argc, char** argv)
{
int myid, numprocs, source;
MPI_Status status;   char message[100];
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myid);
MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
if (myid != 0)  /* 其他进程，向0进程发送HelloWorld信息*/
{   strcpy(message, “Hello World!”);
    MPI_Send(message,strlen(message)+1, MPI_CHAR, 0,99,MPI_COMM_WORLD);
} else  /* 0进程负责从其他进程接受信息并输出*/
{   for (source = 1; source < numprocs; source++)
    {    MPI_Recv(message, 100, MPI_CHAR, source, 99,MPI_COMM_WORLD, &status);
         printf("I am process %d. I recv string '%s' from process %d.\n", myid,message,source);
    }
}
MPI_Finalize();
}
```

```
I am process 0. I recv string ‘Hello World’ from process 1.
I am process 0. I recv string ‘Hello World’ from process 2.
I am process 0. I recv string ‘Hello World’ from process 3.
```



MPI编程示例：消息传递示例2

- 计算大数组元素的开平方之和
- 设系统中共有5个进程，进程号：0，1，2，3，4
- 0号进程作主节点，负责分发数据，不参加子任务计算
- 1-4号进程作为子节点从主进程接受数组数据：
- #1: data[0,4,8,...]
- #2: data[1,5,9,...]
- #3: data[2,6,10,...]
- #4: data[3,7,11,...]

各自求开平方后累加=>本地SqrtSum

- #0: Sqrt Sum = ∑ 各子进程的SqrtSum



MPI编程示例：消息传递示例2

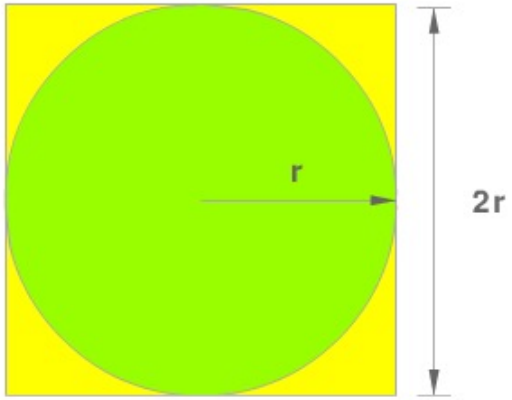

```
#include <stdio.h>   #include <mpi.h>   #include <math.h>   #define N 1002
int main(int argc, char** argv)
{
int myid, P, source, C=0; double  data[N],  SqrtSum=0.0;
MPI_Status status;   char message[100];
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myid);
MPI_Comm_size(MPI_COMM_WORLD,&numprocs);   - -numprocs; /*数据分配时除去0号主节点*/
if (myid== 0){   /*0号主节点，主要负责数据分发和结果收集*/
   for (int  i = 0;  i < N; ++i )  /*数据分发: 0, */
        MPI_Send(data[i], 1, MPI_DOUBLE, i%numprocs+1, 1 ,MPI_COMM_WORLD);
   for (int  source = 1;  source <= numprocs;  ++source)  /*结果收集*/
   {  MPI_Recv(&d, 1, MPI_DOUBLE, source, 99, MPI_COMM_WORLD, &status);  SqrtSum += d; }
}else
{  for (i = myid-1; i < N; i=i+numprocs)  /*各子节点接受数据计算开平方，本地累加*/
   {    MPI_Recv(&d, 1, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD, &status); SqrtSum+=sqrt(d); }
     MPI_Send(SqrtSum, 1, MPI_DOUBLE, 0, 99, MPI_COMM_WORLD); /*本地累加结果送回主节点*/
}
printf("I am process %d. I recv total %d from process 0, and SqrtSum=%f.\n", myid, C, SqrtSum);
MPI_Finalize();
}
```

MPI编程示例：消息传递示例3

- Monte Carlo方法计算圆周率
- Monte Carlo是一种随机抽样统计方法，可用于解决难以用数学公式计算结果的复杂问题近似求解。

作一个直径为2r的圆及其外切正方形，在其中随机产生n个点，落在圆内的点数记为m。根据概率理论，当随机点数足够大时，m与n的比值可近似看成是圆与正方形面积之比。

故有：$m/n \approx \pi \times r^2 / (2r)^2$，$\pi \approx 4m/n$



- 设r取值为0.5，为了提高π计算精度，需要计算尽量大的随机点数，我们考虑在一个并行系统中让每台机器都各自算一个π，然后汇总求一个平均值



MPI编程示例：消息传递示例3


```
#include ”mpi.h”
#include <stdio.h>
#include <stdlib.h>
int main(int argc,char **argv)
{
   int myid, numprocs;
   int namelen,source;
   long count=1000000;
   MPI_Status status;
   MPI_Init(&argc,&argv);
   MPI_Comm_rank(MPI_COMM_WORLD,&myid);
   MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
   srand(myid +(int)time(0));   /* 设置随机种子*/
   double y, x, pi=0.0, n=0.0;
   long m=0,m1=0,i=0,p=0;
   for(i=0;i<count;i++) /* 随机产生一个点(x, y)，判断并计算落在圆内的次数*/
   {  x=(double)rand()/(double)RAND_MAX;
      y=(double)rand()/(double)RAND_MAX;
      if((x-0.5)*(x-0.5)+(y-0.5)*(y-0.5)<0.25)  ++m;
   }
```



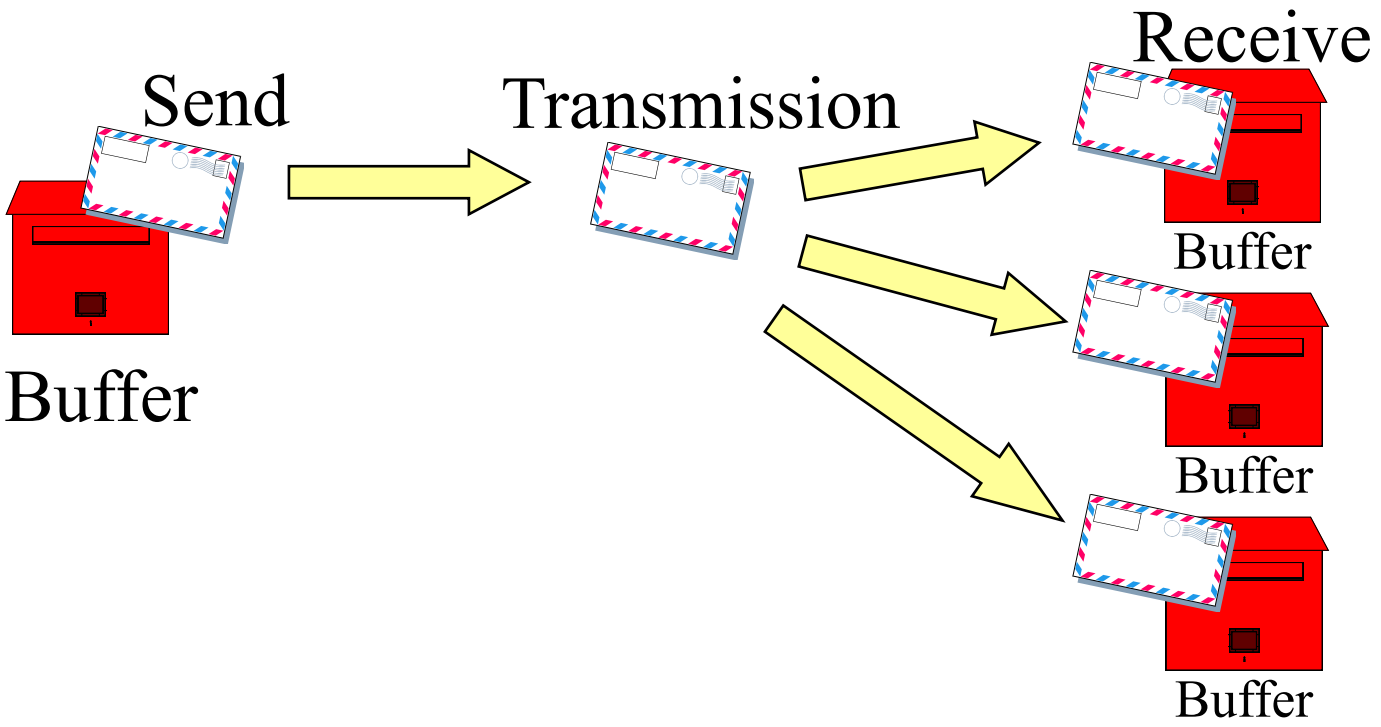
MPI编程示例：消息传递示例3


```
  pi=4.0*m/count;
  printf("Process %d of % pi= %f\n", myid, numprocs, pi);
  if(myid!=0)    /* 从节点将本地计算的π结果发送到主节点*/
  { MPI_Send(&m,1,MPI_DOUBLE,0,1,MPI_COMM_WORLD);  }
  else           /* 主节点接受各从节点的结果并累加*/
  {  p=m;
     for(source=1;source<numprocs;source++)
     {  MPI_Recv(&m1,1,MPI_DOUBLE,source,1,MPI_COMM_WORLD,&status);
        p=p+m1;
     }
     printf("pi= %f\n",4.0*p/(count*numprocs)); /* 各节点输出结果*/
  }
  MPI_Finalize();
}
```

```
Process 0 of 3 pi=3.14135
Process 1 of 3 pi=3.14312
Process 2 of 3 pi=3.14203
pi=3.14216  ←汇总平均值
```

节点集合通信接口

- 提供一个进程与多个进程间同时通信的功能





节点集合通信接口

- 三种类型的集合通信功能

1.同步(Barrier)

MPI_Barrier： 设置同步障使所有进程的执行同时完成

2. 数据移动(Data movement)

MPI_BCAST: 一对多的广播式发送

MPI_GATHER： 多个进程的消息以某种次序收集到一个进程

MPI_SCATTER： 将一个信息划分为等长的段依次发送给其它进程

3. 数据规约(Reduction)

MPI_Reduce： 将一组进程的数据按照指定的操作方式规约到一起并传送给一个进程



节点集合通信接口

- 数据规约操作

  将一组进程的数据按照指定的操作方式规约到一起并传送给一个进程

  MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm)

- 其中规约操作op可设为下表定义的操作之一：

MPI_MAX	求最大值	MPI_MIN	求最小值
MPI_SUM	求和	MPI_PROD	求积
MPI_LAND	逻辑与	MPI_BAND	按位与
MPI_LOR	逻辑或	MPI_BOR	按位或
MPI_LXOR	逻辑异或	MPI_BXOR	按位异或
MPI_MAXLOC	最大值和位置	MPI_MINLOC	最小值和位置

节点集合通信接口

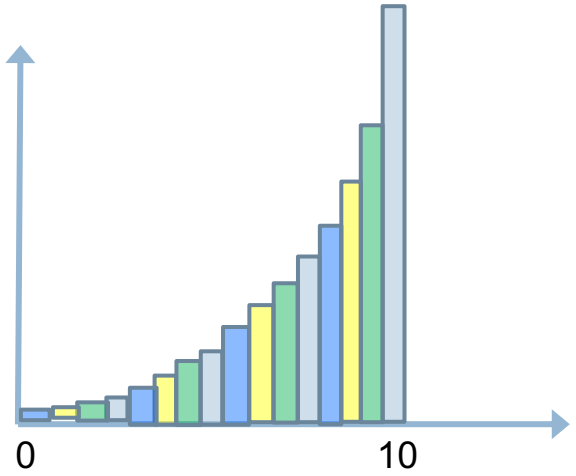
规约操作编程示例-计算积分

根据微积分原理，任一函数f(x)在区间[a,b]上的积分是由各个x处的y值为高构成的N个小矩形(当N趋向无穷大时的）面积之和构成。因此,选取足够大的N可近似计算积分。

设$y=x^2$，求其在[0,10]区间的积分。

先把[0,10]分为N个小区间，则对每个x取值对应小矩形面积为：y*10/N

求和所有矩形面积，当N足够大时即为近似积分值。



我们用n个节点来分工计算N个区间的面积。如图所示，根据总结点数目，每个节点将求和一个颜色的小矩形块。



节点集合通信接口


```
#define N 100000000
#define  a 0
#define  b 10
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include “mpi.h”
int main(int argc, char** argv)
{
  int myid,numprocs;
  int i;
  double local=0.0, dx=(double)(b-a)/N; /* 小矩形宽度 */
  double inte, x;
  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &myid);
  MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
```



节点集合通信接口


```
for(i=myid;i<N;i=i+numprocs)  /* 根据节点数目将N个矩形分为图示的多个颜色组 */
  {  /* 每个节点计算一个颜色组的矩形面积并累加*/
     x = a + i*dx +dx/2; /* 以每个矩形的中心点x值计算矩形高度 */
     local +=x*x*dx;  /* 矩形面积 = 高度x宽度=y*dx */
  }
  MPI_Reduce(&local,&inte,1,MPI_DOUBLE,MPI_SUM, 0, MPI_COMM_WORLD);
  if(myid==0)                          /* 规约所有节点上的累加和并送到主节点0 */
  { /* 主节点打印累加和*/
    printf("The integal of x*x in region [%d,%d] =%16.15f\n", a, b, inte);
  }
  MPI_Finalize();
}
```

The integal of x*x in region[0, 10] = 333.33345



MPI的特点和不足

- **MPI的特点**
  - 灵活性好，适合于各种计算密集型的并行计算任务
  - 独立于语言的编程规范，可移植性好
  - 有很多开放机构或厂商实现并支持
- **MPI的不足**
  - 无良好的数据和任务划分支持
  - 缺少分布文件系统支持分布数据存储管理
  - 通信开销大，当计算问题复杂、节点数量很大时，难以处理，性能大幅下降
  - 无节点失效恢复机制，一旦有节点失效，可能导致计算过程无效
  - 缺少良好的构架支撑，程序员需要考虑以上所有细节问题，程序设计较为复杂



摘要

- 为什么需要并行计算?
- 并行计算技术的分类
- 并行计算的主要技术问题
- MPI并行程序设计
- 为什么需要大规模数据并行处理?

为什么需要海量数据并行处理技术?

- 处理数据的能力大幅落后于数据增长，需要寻找有效的数据密集型并行计算方法

磁盘容量增长远远快过存储访问带宽和延迟：80年代中期数十MB到今天1-2TB，增长10万倍，而延迟仅提高2倍，带宽仅提高50倍!

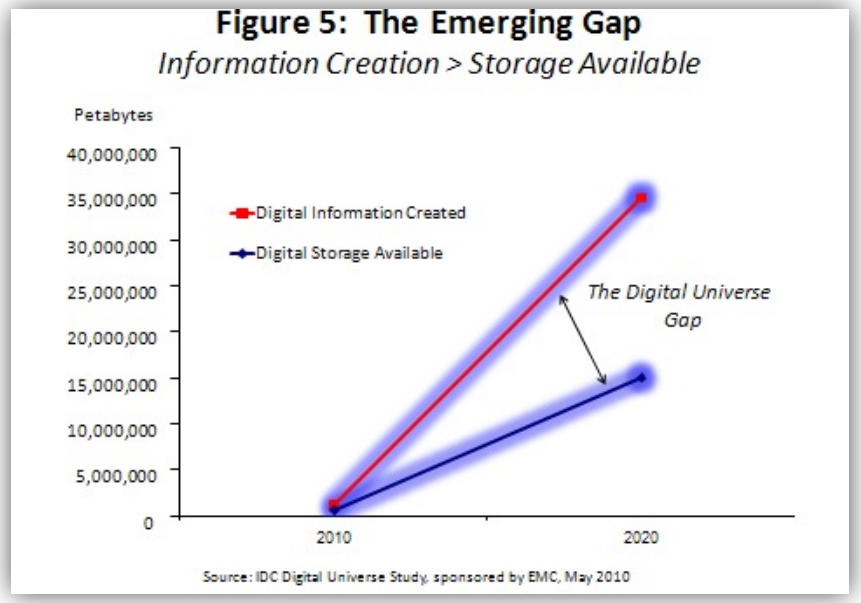
- 100TB数据顺序读一遍需要多少时间?

设硬盘读取访问速率128MB/秒

1TB/128MB 约2.17小时

100TB/128MB = 217小时 = 9天!

即使用百万元高速磁盘阵列(800MB/s)，仍需1.5天!



为什么需要海量数据并行处理技术?

- 海量数据隐含着更准确的事实

  信息检索、自然语言理解和机器学习的三个要素:

  数据，特征，算法

- 2001，Banko and Brill 发表了一篇自然语言领域的经典研究论文，探讨训练数据集大小对分类精度的影响，发现数据越大，精度越高；更有趣的发现是，他们发现当数据不断增长时，不同算法的分类精度趋向于相同，使得小数据集时不同算法在精度上的差别基本消失!
- 结论引起争论：算法不再要紧，数据更重要！不再需要研究复杂算法，找更多数据就行了!

为什么需要海量数据并行处理技术?

- 海量数据隐含着更准确的事实
  - 2001年，一个基于事实的简短问答研究，如提问：Who shot Abraham Lincoln? 在很大的数据集时，只要使用简单的模式匹配方法，找到在“shot Abraham Lincoln”前面的部分即可快速得到准确答案：John Wilkes Booth
  - 2007， Brants et al. 描述了一个基于2万亿个单词训练数据集的语言模型，比较了当时最先进的Kneser-Ney smoothing 算法与他们称之为“stupid backoff “ (愚蠢退避)的简单算法，最后发现，后者在小数据集时效果不佳，但在大数据集时，该算法最终居然产生了更好的语言模型!
- 结论：大数据集上的简单算法能比小数据集上的复杂算法产生更好的结果!



为什么需要MapReduce?

- 并行计算技术和并行程序设计的复杂性

  依赖于不同类型的计算问题、数据特征、计算要求、和系统构架，并行计算技术较为复杂，程序设计需要考虑数据划分，计算任务和算法划分，数据访问和通信同步控制，软件开发难度大，难以找到统一和易于使用的计算框架和编程模型与工具。

- 海量数据处理需要有效的并行处理技术

  海量数据处理时，依靠MPI等并行处理技术难以凑效

- MapReduce是目前面向海量数据处理最为成功的技术

  MapReduce是目前业界和学界公认的最为有效和最易于使用的海量数据并行处理技术

  Google，IBM，Amazon，百度等国内外公司普遍使用



MapReduce简介

- 问题与需求：如何对巨量的Web文档建立索引、根据网页链接计算网页排名，从上百万文档中训练垃圾邮件过滤器，运行气象模拟，数十亿字符串的排序?
- 解决方案：如果你想学习如果编写程序完成这些巨量数据的处理问题，MapReduce将为你提供一个强大的分布式计算环境和构架，让你仅需关注你的应用问题本身，编写很少的程序代码即可完成看似难以完成的任务!
- 什么是MapReduce? MapReduce是Google公司发明的一种面向大规模海量数据处理的高性能并行计算平台和软件编程框架，是目前最为成功和最易于使用的大规模海量数据并行处理技术，广泛应用于搜索引擎（文档倒排索引，网页链接图分析与页面排序等）、Web日志分析、文档分析处理、机器学习、机器翻译等各种大规模数据并行计算应用领域。



MapReduce简介

MapReduce是面向大规模数据并行处理的：

- 基于集群的高性能并行计算平台(Cluster Infrastructure)

  允许用市场上现成的普通PC或性能较高的刀架或机架式服务器，构成一个包含数千个节点的分布式并行计算集群

- 并行程序开发与运行框架(Software Framework)

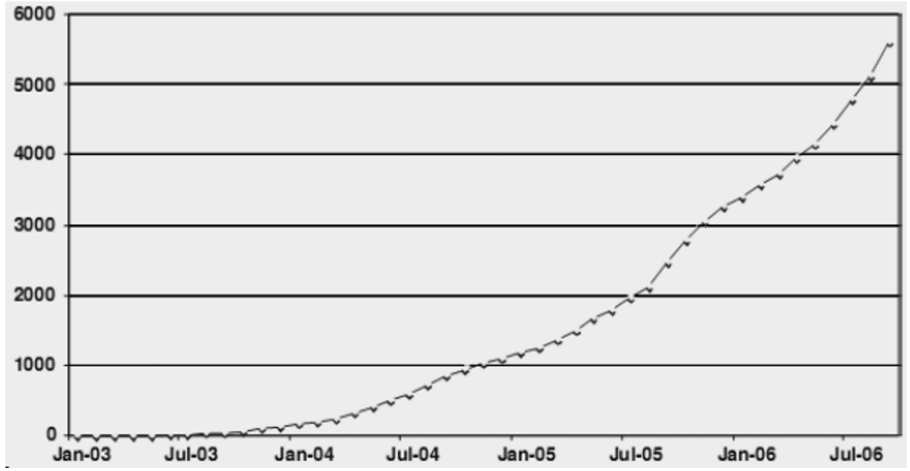
  提供了一个庞大但设计精良的并行计算软件构架，能自动完成计算任务的并行化处理，自动划分计算数据和计算任务，在集群节点上自动分配和执行子任务以及收集计算结果，将数据分布存储、数据通信、容错处理等并行计算中的很多复杂细节交由系统负责处理，大大减少了软件开发人员的负担

- 并行程序设计模型与方法(Programming Model & Methodology)

  借助于函数式语言中的设计思想，提供了一种简便的并行程序设计方法，用Map和Reduce两个函数编程实现基本的并行计算任务，提供了完整的并行编程接口，完成大规模数据处理

Google MapReduce

- 2004年，Google在OSDI国际会议上发表了一篇论文：*“MapReduce: Simplified Data Processing on Large Clusters”*，公布了MapReduce的基本原理和主要设计思想。
- Google公司用MapReduce重新改写了其整个搜索引擎中的Web文档索引处理
- 自MapReduce发明后，Google大量用于各种海量数据处理，2006年7月统计Google内部有7千以上的程序基于MapReduce实现





Google MapReduce

- 巨大的Web搜索引擎和海量数据并行处理硬件平台

  Google目前在全球的数十个数据中心使用了百万台以上的服务器构成其强大的Web搜索和海量数据并行计算平台，支撑其搜索引擎、Gmail、Google Map、Google Earth、以及其云计算平台AppEngine的大型应用服务需求。

- 强大的数据处理服务能力

  Google可提供超过80亿网页和10亿张图片的检索索引，每天处理2亿次以上检索请求，平均每个检索耗时0.5秒；每个搜索请求背后有上千个服务器同时进行检索计算和服务



Hadoop MapReduce简介

- 在Google发表了文章后，Doug Cutting，2004年，开源项目Lucene(搜索索引程序库)和Nutch(搜索引擎)的创始人，发现MapReduce正是其所需要的解决大规模分布数据处理的重要技术，因而模仿Google MapReduce，基于Java设计出了称为Hadoop的开源MapReduce，该项目成为Apache下最重要的项目
- Apache Hadoop目前的最新版本是3.3.4 （Aug 8, 2022）



为什么MapReduce很重要?

- Why is MapReduce important? In practical terms, it provides a very effective tool for tackling large-data problems.
- But beyond that, MapReduce is important in how it has changed the way we organize computations at a massive scale.
- MapReduce represents the first widely-adopted step away from the von Neumann model. MapReduce can be viewed as the first breakthrough in the quest for new abstractions that allow us to organize computations, not over individual machines, but over entire clusters.
- MapReduce is the most successful abstraction over large-scale computational resources we have seen to date.

Cite from Jimmy Lin, University of Maryland, Data-Intensive Text processing with MapReduce，2010

THANK YOU



南京大学
NANJING UNIVERSITY

南京大学计算机软件研究所
Institute of Computer Software, Nanjing University